

L03a · CORALNPU 教学仓

L03a · 流水线与冒险

作业说明 —— 任务拆解、验收标准与调试方法

对应上游仓库: [google/coralnpu](#) (RTL、文档、测试程序都来自这里)

学习方式: 读规格 → 写 RTL → `./learn check` 对拍 → 修复

文档版本: 2026-09-13 · 生成命令: `./learn pdf`

目录

L03a 作业说明

0. 交付物
1. 接口契约
2. 六个 TODO
3. 验收标准
4. 调试方法
5. 常见坑（都是本课作者踩过的）
6. 进阶挑战（不计入验收）
7. 学习记录（建议提交）

L03a 作业说明

0. 交付物

只改一个文件： `course/lessons/L03a_pipeline/rtl/pipeline_core.sv` (6 个 TODO)。

复用了前面的成果：

- L02 的 `lsu.sv` (你的实现) —— 流水线把访存请求交给它；
- L00 的 `alu32.sv` (你的实现) —— EX 级的运算；
- L02 的测试平台 `tb_cpu.sv` (支持 `+LATENCY`) 和 L01/L02 的全部测试程序。

验收：

```
./learn check L03a           # 期望 12/12 (6 个程序 × 2 种存储器延迟)
./learn check L03a --only 05 # 只看某个程序，并打印 CPI
./learn check L03a --ref      # 参考实现
```

1. 接口契约

端口与 L02 的 `cpu_core` 完全一样 (`io_imem_*`、`io_dmem_*` 含 `valid/ready`、`o_retire_*`、`o_halted`)。新增的内部约定：

1. **trace 在 WB 级产生**： `o_retire_valid` 只在 `wb_valid` 且没有整条流水线冻结时为 1；同一条指令只能出现一次；
2. **访存指令在 MEM 级发请求**： `lsu_req_valid = mem_op & ~lsu_busy`；
3. **停机指令** (`mpause/ebreak/ecall`) 在 EX 级触发“停止取指”，然后正常流到 WB 退休。

2. 六个 TODO

TODO 1: 旁路 (forwarding)

```
always_comb begin
    fwd_a = ex_rs1_val;
    fwd_b = ex_rs2_val;
    // MEM 级更新，优先；但 MEM 级是 load 时数据还没出来 (交给 TODO 3)
    if (mem_valid && mem_reg_we && !mem_is_load && rd_of(mem_inst) == rs1_of(ex_inst) && rd_of(mem_inst) != 0)
        fwd_a = mem_alu_y;
    else if (wb_valid && wb_reg_we && rd_of(wb_inst) == rs1_of(ex_inst) && rd_of(wb_inst) != 0)
        fwd_a = wb_data;
    // rs2 同理
end
```

TODO 2: WB→ID 写穿

ID 读寄存器堆时，如果 WB 级正在写同一个寄存器，要用 `wb_data` 顶上。（这一步不做，会看到”某条指令读到的值比预期旧一拍”，而且只在特定指令距离下出错。）

TODO 3: load-use 停顿

```
load_use_stall = ex_valid & ex_is_load & id_valid & (rd_of(ex_inst) != 0) &
                ((uses_rs1_of(id_inst) && rs1_of(id_inst) == rd_of(ex_inst)) ||
                 (uses_rs2_of(id_inst) && rs2_of(id_inst) == rd_of(ex_inst)));
```

停顿行为：冻结 PC/IF，给 ID/EX 插气泡，EX/MEM 与 MEM/WB 继续前进。

TODO 4: 控制冒险（冲刷 + 重定向）

```
ex_redirect = ex_valid & (ex_br_taken | ex_is_jal | ex_is_jalr | ex_is_halt);
```

效果：`pc <= ex_target`，IF/ID 与 ID/EX 的有效位置 0。

TODO 5: 气泡的控制位清零

```
mem_reg_we  <= ex_reg_we  & ex_valid;
mem_is_load <= ex_is_load & ex_valid;
mem_is_store <= ex_is_store & ex_valid;
```

这一条最容易漏：忘了它，被冲刷的指令会带着上一条指令的 `reg_we` 一路写到寄存器堆，表现为”某个无关寄存器被莫名改成别的值”。

TODO 6: 结构冒险（等 LSU）

`stall_all = lsu_wait` 时冻结**整条流水线（含 MEM/WB）**。原因见《基础知识》第 4 章：停在 EX 的消费者可能还要用 WB 级的结果做旁路。

3. 验收标准

```
[通过] 01_arith.S   lat0 / lat2
[通过] 02_branch.S lat0 / lat2
[通过] 03_mem.S     lat0 / lat2
[通过] 04_call.S   lat0 / lat2
[通过] 05_c_fib.c  lat0 / lat2
[通过] 06_unaligned.S lat0 / lat2
```

12/12 次运行通过

4. 调试方法

第一招：看失败模式。

检查器说	通常是
第 2 条指令就错（如 <code>addi sp, sp, -16</code> ）	旁路完全没接（TODO 1）
某条指令读到的值旧一拍	WB→ID 写穿没做（TODO 2）
<code>load</code> 后紧跟的指令错	load-use 停顿没做（TODO 3）
跳转/分支之后 trace 全乱	冲刷/重定向没做（TODO 4）
“某个无关寄存器被改动”	气泡控制位没清零（TODO 5）
超时、且波形里 stall 恒为 1	停顿条件写成了死循环（TODO 3/6）

第二招：把流水线占用打出来。

```
`ifdef DEBUG
always_ff @(posedge clk) if (!rst)
    $display("IF=%05x ID=%05x(%b) EX=%05x(%b) MEM=%05x(%b) WB=%05x(%b)",
            pc, id_pc, id_valid, ex_pc, ex_valid, mem_pc, mem_valid, wb_pc, wb_valid);
`endif
```

第三招：写最小复现。 一个 `addi` + `add` 的两条指令程序就能验证旁路；一个 `lw` + `add` 验证 load-use；一个 `beq` 跳自己验证冲刷。

5. 常见坑（都是本课作者踩过的）

现象	根因	修法
第二条指令就少加了一位（如 <code>sp</code> 差 16）	旁路没做，读到旧寄存器	TODO 1
程序能跑一段时间后在循环里错	WB 级结果被“写穿”漏掉	TODO 2
无条件死循环、stall 恒 1	load-use 时把整条流水线冻住了	只用 <code>stall_bubble</code> 冻 PC/IF
结果偶尔错、寄存器被莫名改写	气泡带控制位进来写寄存器堆	TODO 5
长程序错、短程序对	等 LSU 时没冻 MEM/WB，旁路来源跑掉	用 <code>stall_all</code> 冻全部
trace 行数比期望多几行	停顿期间 <code>o_retire_valid</code> 仍为 1	retire 用 <code>~stall_all</code> 门控

6. 进阶挑战（不计入验收）

- **挑战 1（降 CPI）：** `05_c_fib.c` 的 CPI 是 2.66。想办法把它降到 2.2 以下，提示：分支/jal 在 ID 级就能解析（`jal` 的目标不依赖操作数），可以少冲刷一条；
- **挑战 2（静态预测）：** 实现“后向分支预测跳转”（像上游那样），看 CPI 变化；
- **挑战 3（LSU 不停全流水）：** 让访存停顿只影响需要访存的指令（提示：需要记分板/重排序缓冲，这正是上游 `RetirementBuffer.scala` 做的事）；

- **挑战 4（形态）**：把 5 级改成 4 级（像上游那样把 MEM 并进 EX），对比 CPI 与关键路径。

7. 学习记录（建议提交）

在 `course/work/L03a/记录.md` 里写下：

1. 你先做出的错误版本是什么现象，最后定位到哪一条；
2. 两个 CPI 数字（L02 与 L03a）分别多少，你认为差距主要来自哪里；
3. 对照上游 4 级流水，说出你的实现比它”少”了什么。